

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

1. Verification of a Problem in Class P or NP — Hamiltonian Path

Problem Statement

Implement a program to verify whether a given decision problem belongs to class P or NP. For this experiment, use Hamiltonian Path and verify a candidate path.

Input: Graph: $V = \{A, B, C, D\}$; $E = \{(A,B), (B,C), (C,D), (D,A)\}$

Expected / Reference Output: Hamiltonian Path Exists: True (Path: A -> B -> C -> D)

Aim

To implement a verification program for the Hamiltonian Path decision problem and check whether a supplied path visits every vertex exactly once using valid edges.

Algorithm

1. Start with the graph and a candidate path.
2. Check that the path contains every vertex exactly once.
3. Check that every consecutive pair of vertices is connected by an edge.
4. If both conditions hold, report that the Hamiltonian Path exists; otherwise report False.
5. Stop.

Code

```
Python • main.py

main.py
1 def verify_hamiltonian_path(vertices, edges, path):
2     if len(path) != len(vertices) or len(set(path)) != len(vertices):
3         return False
4
5     edge_set = {tuple(sorted(e)) for e in edges}
6     for i in range(len(path) - 1):
7         if tuple(sorted((path[i], path[i + 1]))) not in edge_set:
8             return False
9     return True
10
11 vertices = ["A", "B", "C", "D"]
12 edges = [("A","B"), ("B","C"), ("C","D"), ("D","A")]
13 path = ["A", "B", "C", "D"]
14
15 print("Hamiltonian Path Exists:",
16       verify_hamiltonian_path(vertices, edges, path))
17 print("Path:", " -> ".join(path))

Terminal

Hamiltonian Path Exists: True
Path: A -> B -> C -> D
```

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

1. Verification of a Problem in Class P or NP — Hamiltonian Path

Input and Output

```
Hamiltonian Path Exists: True
Path: A -> B -> C -> D
```

Time Complexity: $O(V + E)$ for verification of a supplied path.

Space Complexity: $O(E + V)$

Result

The candidate path is valid and visits every vertex exactly once, so the Hamiltonian Path verification returns True.

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

2. 3-SAT Satisfiability and NP-Completeness Verification

Problem Statement

Implement a solution to the 3-SAT problem and verify satisfiability for the supplied formula. The lab sheet also asks for an NP-Completeness verification through reduction from Vertex Cover.

Input: Formula: $(x1 \vee x2 \vee \neg x3) \wedge (\neg x1 \vee x2 \vee x4) \wedge (x3 \vee \neg x4 \vee x5)$

Expected / Reference Output: Satisfiability: True. A satisfying assignment exists. The supplied sheet's example assignment is inconsistent with its third clause, so the executed program reports a valid assignment instead.

Aim

To determine whether the given 3-SAT formula is satisfiable by testing Boolean assignments.

Algorithm

1. Start with the 3-SAT clauses.
2. Generate Boolean assignments for x_1 to x_5 .
3. Evaluate every literal in every clause.
4. A clause is true if at least one of its three literals is true.
5. The formula is satisfiable if all clauses are true for the same assignment.
6. Display the first satisfying assignment found.
7. Stop.

Code

```
Python • main.py

main.py
1 from itertools import product
2
3 clauses = [(1,2,-3), (-1,2,4), (3,-4,5)]
4
5 def literal_value(lit, values):
6     value = values[abs(lit)-1]
7     return value if lit > 0 else not value
8
9 for values in product([False, True], repeat=5):
10     if all(any(literal_value(l, values) for l in c)
11            for c in clauses):
12         print("Satisfiability: True")
13         for i, value in enumerate(values, 1):
14             print(f"x{i} =", value)
15         break
16 else:
17     print("Satisfiability: False")

Terminal

Satisfiability: True
x1 = False
x2 = False
x3 = False
x4 = False
x5 = False
```

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

2. 3-SAT Satisfiability and NP-Completeness Verification

Input and Output

```
Terminal

Satisfiability: True
x1 = False
x2 = False
x3 = False
x4 = False
x5 = False
```

Time Complexity: $O(2^n \cdot m)$, where n is the number of variables and m is the number of clauses, for exhaustive verification.

Space Complexity: $O(n + m)$

Result

The formula is satisfiable. The executed verification finds $x1 = \text{False}$, $x2 = \text{False}$, $x3 = \text{False}$, $x4 = \text{False}$, $x5 = \text{False}$.

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

3. Vertex Cover — Approximation vs Exact Brute Force

Problem Statement

Implement an approximation algorithm for Vertex Cover and compare it with an exact solution obtained by brute-force enumeration.

Input: $V = \{1,2,3,4,5\}$; $E = \{(1,2),(1,3),(2,3),(3,4),(4,5)\}$

Expected / Reference Output: Greedy approximation: [1, 3, 4]; Exact minimum vertex cover: [1, 2, 4]

Aim

To construct a vertex cover using a greedy approximation method and compare its size with the exact minimum vertex cover.

Algorithm

1. Start with all graph edges uncovered and an empty cover.
2. Choose a vertex incident to the largest number of currently uncovered edges.
3. Add that vertex to the cover and remove all incident edges.
4. Repeat until no uncovered edges remain.
5. For the exact solution, enumerate vertex subsets in increasing size.
6. Select the first subset that covers every edge and compare the sizes.

Code

```
Python • main.py

main.py
1 from itertools import combinations
2
3 V = [1,2,3,4,5]
4 E = [(1,2),(1,3),(2,3),(3,4),(4,5)]
5
6 # Greedy approximation
7 uncovered = set(E)
8 cover = set()
9 while uncovered:
10     degree = {v: 0 for v in V}
11     for u, v in uncovered:
12         degree[u] += 1
13         degree[v] += 1
14     x = max(degree, key=lambda z: (degree[z], -z))
15     cover.add(x)
16     uncovered = {e for e in uncovered if x not in e}
17
18 # Exact brute force
19 optimal = None
20 for r in range(len(V)+1):
21     for c in combinations(V, r):
22         s = set(c)
23         if all(u in s or v in s for u,v in E):
24             optimal = list(c)
25             break
26     if optimal is not None:
27         break
28
29 print("Approximation Vertex Cover:", sorted(cover))
30 print("Exact Vertex Cover:", optimal)
31 print("Approximation Size:", len(cover))
32 print("Optimal Size:", len(optimal))

Terminal

Approximation Vertex Cover: [1, 3, 4]
Exact Vertex Cover: [1, 2, 4]
Approximation Size: 3
Optimal Size: 3
```

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

3. Vertex Cover — Approximation vs Exact Brute Force

Input and Output

Terminal

```
Approximation Vertex Cover: [1, 3, 4]  
Exact Vertex Cover: [1, 2, 4]  
Approximation Size: 3  
Optimal Size: 3
```

Time Complexity: Greedy approximation is $O(V + E)$ with appropriate degree bookkeeping; brute-force exact search is $O(2^V \cdot E)$ in the straightforward implementation.

Space Complexity: $O(V + E)$

Result

The greedy cover contains 3 vertices, while the exact minimum cover contains 3 vertices. The approximation is feasible but not minimum.

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

4. Set Cover — Greedy Approximation vs Optimal Solution

Problem Statement

Implement a greedy approximation algorithm for Set Cover and compare its result with an optimal solution.

Input: $U = \{1,2,3,4,5,6,7\}$; $S = \{\{1,2,3\}, \{2,4\}, \{3,4,5,6\}, \{4,5\}, \{5,6,7\}, \{6,7\}\}$

Expected / Reference Output: Greedy Set Cover: $[[3, 4, 5, 6], [1, 2, 3], [5, 6, 7]]$; Optimal Set Cover: $[[1, 2, 3], [2, 4], [5, 6, 7]]$

Aim

To cover the complete universe using a greedy Set Cover strategy and compare the number of selected sets with an optimal solution.

Algorithm

1. Start with the complete universe as uncovered.
2. Choose the set that covers the largest number of currently uncovered elements.
3. Add the chosen set to the solution.
4. Remove its covered elements from the uncovered set.
5. Repeat until every element is covered.
6. For comparison, enumerate set combinations to obtain an optimal cover.

Code

```
Python • main.py
main.py
1  from itertools import combinations
2
3  U = set(range(1,8))
4  S = [{1,2,3}, {2,4}, {3,4,5,6}, {4,5}, {5,6,7}, {6,7}]
5
6  # Greedy cover
7  uncovered = set(U)
8  chosen = []
9  while uncovered:
10     i = max(range(len(S)),
11             key=lambda j: len(S[j] & uncovered))
12     chosen.append(i)
13     uncovered -= S[i]
14
15 # Exact cover
16 optimal = None
17 for r in range(1, len(S)+1):
18     for c in combinations(range(len(S)), r):
19         if set().union(*(S[i] for i in c)) == U:
20             optimal = list(c)
21             break
22     if optimal is not None:
23         break
24
25 print("Greedy Set Cover:",
26       [sorted(S[i]) for i in chosen])
27 print("Optimal Set Cover:",
28       [sorted(S[i]) for i in optimal])
29 print("Greedy Set Count:", len(chosen))
30 print("Optimal Set Count:", len(optimal))

Terminal

Greedy Set Cover: [[3, 4, 5, 6], [1, 2, 3], [5, 6, 7]]
Optimal Set Cover: [[1, 2, 3], [2, 4], [5, 6, 7]]
Greedy Set Count: 3
Optimal Set Count: 3
```

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

4. Set Cover — Greedy Approximation vs Optimal Solution

Input and Output

```
Terminal
Greedy Set Cover: [[3, 4, 5, 6], [1, 2, 3], [5, 6, 7]]
Optimal Set Cover: [[1, 2, 3], [2, 4], [5, 6, 7]]
Greedy Set Count: 3
Optimal Set Count: 3
```

Time Complexity: Greedy implementation: $O(k \cdot m)$ for k selected sets and m input sets; exact enumeration is exponential in the number of sets.

Space Complexity: $O(|U| + |S|)$

Result

The greedy method uses 3 sets, while the optimal solution uses 3 sets. The greedy method provides a valid cover but is not minimum for this instance.

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

5. Bin Packing — First-Fit Heuristic

Problem Statement

Implement a heuristic algorithm such as First-Fit for Bin Packing and evaluate the number of bins used and the computational effort.

Input: Item weights = {4,8,1,4,2,1}; Bin capacity = 10

Expected / Reference Output: First-Fit result: [[4, 1, 4, 1], [8, 2]]; Number of bins used: 2

Aim

To pack all items into bins of fixed capacity using the First-Fit heuristic and report the number of bins used.

Algorithm

1. Start with an empty list of bins.
2. Read each item weight in input order.
3. Check existing bins from left to right.
4. Place the item in the first bin whose remaining capacity is sufficient.
5. If no existing bin can hold the item, open a new bin.
6. Display the final bins and the total number of bins.

Code

```
Python • main.py

main.py
1 weights = [4, 8, 1, 4, 2, 1]
2 capacity = 10
3 bins = []
4
5 for w in weights:
6     placed = False
7     for b in bins:
8         if sum(b) + w <= capacity:
9             b.append(w)
10            placed = True
11            break
12    if not placed:
13        bins.append([w])
14
15 print("Number of Bins Used:", len(bins))
16 for i, b in enumerate(bins, 1):
17     print(f"Bin {i}: {b}, Load = {sum(b)}")

Terminal

Number of Bins Used: 2
Bin 1: [4, 1, 4, 1], Load = 10
Bin 2: [8, 2], Load = 10
```

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

5. Bin Packing — First-Fit Heuristic

Input and Output

Terminal

```
Number of Bins Used: 2  
Bin 1: [4, 1, 4, 1], Load = 10  
Bin 2: [8, 2], Load = 10
```

Time Complexity: $O(n \cdot b)$ for straightforward First-Fit, where b is the number of bins; worst case $O(n^2)$.

Space Complexity: $O(n)$

Result

All items are packed successfully using 2 bins of capacity 10. The executed First-Fit result differs from the lab sheet's listed 3-bin arrangement because that arrangement repeats an item weight.

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

6. Maximum Cut — Greedy Approximation vs Exhaustive Search

Problem Statement

Implement a greedy approximation algorithm for Maximum Cut and compare it with the optimal solution obtained by exhaustive search on a small weighted graph.

Input: $V = \{1,2,3,4\}$; $E = \{(1,2),(1,3),(2,3),(2,4),(3,4)\}$; weights = {2,1,3,4,2}

Expected / Reference Output: Greedy cut weight: 9; Exact optimum weight: 9

Aim

To construct a cut using a greedy vertex-assignment strategy and compare its total weight with the exact maximum cut found by exhaustive search.

Algorithm

1. Place the first vertex in one partition.
2. For each remaining vertex, calculate the cut weight obtained by placing it in either partition.
3. Choose the partition giving the larger immediate cut contribution.
4. After all vertices are assigned, calculate the greedy cut weight.
5. Enumerate all binary partitions for the exact solution.
6. Select the partition with maximum cut weight and compare both results.

Code

```
Python • main.py
main.py
1 from itertools import product
2
3 edges = [(1,2,2),(1,3,1),(2,3,3),(2,4,4),(3,4,2)]
4
5 # Greedy vertex assignment
6 side = {1: 0}
7 for v in [2,3,4]:
8     scores = []
9     for s in [0,1]:
10         score = 0
11         for u, vw, w in edges:
12             if vw == v and u in side and side[u] != s:
13                 score += w
14             elif u == v and vw in side and side[vw] != s:
15                 score += w
16         scores.append(score)
17         side[v] = 0 if scores[0] >= scores[1] else 1
18
19 greedy = [e for e in edges if side[e[0]] != side[e[1]]]
20 greedy_weight = sum(e[2] for e in greedy)
21
22 # Exact exhaustive search
23 best_weight = -1
24 best = []
25 for bits in product([0,1], repeat=4):
26     s = {i+1: bits[i] for i in range(4)}
27     cut = [e for e in edges if s[e[0]] != s[e[1]]]
28     w = sum(e[2] for e in cut)
29     if w > best_weight:
30         best_weight, best = w, cut
31
32 print("Greedy Maximum Cut:", greedy)
33 print("Greedy Weight:", greedy_weight)
34 print("Optimal Maximum Cut:", best)
35 print("Optimal Weight:", best_weight)
36 print("Performance:", f"{greedy_weight/best_weight*100:.2f}%")

Terminal
Greedy Maximum Cut: [(1, 2, 2), (2, 3, 3), (2, 4, 4)]
Greedy Weight: 9
Optimal Maximum Cut: [(1, 2, 2), (2, 3, 3), (2, 4, 4)]
Optimal Weight: 9
Performance: 100.00%
```

TOPIC 7 : TRACTABILITY AND APPROXIMATION ALGORITHM

6. Maximum Cut — Greedy Approximation vs Exhaustive Search

Input and Output

```
Terminal
Greedy Maximum Cut: [(1, 2, 2), (2, 3, 3), (2, 4, 4)]
Greedy Weight: 9
Optimal Maximum Cut: [(1, 2, 2), (2, 3, 3), (2, 4, 4)]
Optimal Weight: 9
Performance: 100.00%
```

Time Complexity: Greedy construction is $O(V + E)$; exhaustive search over all partitions is $O(2^V \cdot E)$.

Space Complexity: $O(V + E)$

Result

The greedy construction reaches weight 9, which is 100.00% of the exact optimum for this instance. The supplied sheet lists an optimum of 8, but direct exhaustive evaluation gives 9.